Manufacturing Operational Insights Report

Dockerized Entertainment React Service

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	**Maitry Banduke**	**EN22CS301571**
02	**Mohd Quasim**	**EN22CS301603**
03	**Mrunali Kamerikar**	**EN22CS301618**
04	**Muskan Asija**	**EN22CS301623**

Group Name: G03 D6

Project Number : DO-14

Industry Mentor Name: Vaibhav

University Mentor Name: Akshay Saxena

Academic Year: 2025-2026

1. Introduction

The **Dockerized Entertainment React Service** project focuses on deploying an entertainment web application using modern **DevOps practices and container-based technologies**. Traditional deployment methods often involve manual steps that can lead to configuration errors, inconsistent environments, and scaling issues. To overcome these challenges, the project uses **Docker, Kubernetes, and CI/CD pipelines** to automate the application lifecycle. Docker is used to package the **React-based entertainment application** along with its dependencies into containers, ensuring consistent performance across development, testing, and production environments.

CI/CD tools such as **Jenkins or GitHub Actions** automatically build and test the application whenever code is pushed to the repository. The Docker image is then stored in a container registry like **Docker Hub or GitHub Container Registry**, and **Kubernetes** deploys and manages the containers. Kubernetes provides features such as **auto-scaling, service exposure, and rolling updates**, ensuring reliable and scalable deployment of the application.

1.1 Scope of the document

The scope of this project focuses on the development, containerization, and deployment of an Entertainment React Service using modern DevOps practices. It covers the complete workflow starting from when a developer pushes code to the GitHub repository to the automated processes of building, testing, and deploying the application in a Kubernetes environment. The system is designed to automate the application lifecycle and ensure a reliable and scalable deployment process.

The application is developed using React.js to create a responsive and interactive frontend interface. Git is used as the version control system, allowing developers to manage code changes and collaborate efficiently through a GitHub repository. Whenever new code is pushed to the repository, automated CI/CD pipelines are triggered to start the build and testing processes.

Docker is used to containerize the application along with its dependencies, ensuring that it runs consistently across development, testing, and production environments. The containerized application is then deployed using Kubernetes, which manages the containers, handles scaling, load balancing, and ensures high availability. Tools such as Jenkins or GitHub Actions are used to automate the CI/CD workflow, enabling faster and more reliable application deployment.

1.2 System overview

The **Dockerized Entertainment React Service** is designed to provide a fully automated DevOps pipeline that manages the entire lifecycle of the application from development to deployment. The system integrates version control, automated build processes, containerization, and container orchestration into a single workflow.The process begins when a developer writes or updates the application code and pushes it to a **GitHub repository**. This action automatically triggers a **CI/CD pipeline** configured using Jenkins or GitHub Actions. The pipeline then performs several automated tasks, including compiling the React application, running tests to verify functionality, and packaging the application into a Docker container.

Once the Docker image is created, it is pushed to a **container registry** such as Docker Hub or GitHub Container Registry. Kubernetes retrieves the latest image from the registry and deploys it within the cluster environment. Kubernetes manages the running containers and ensures that the application remains available and responsive.

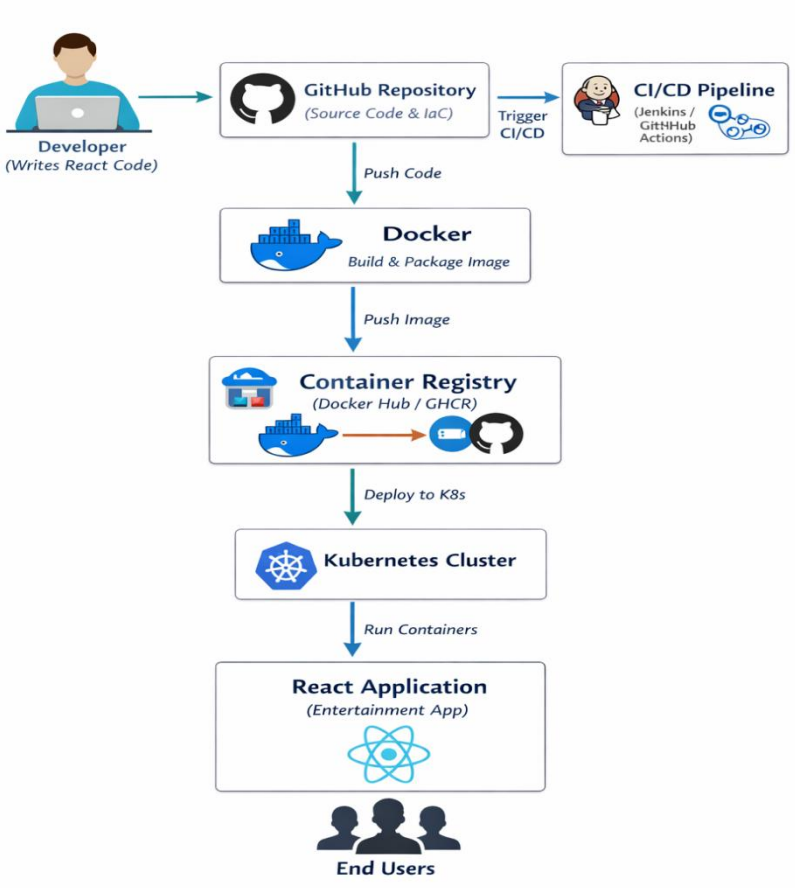
Kubernetes also provides advanced management features such as **container orchestration, horizontal scaling, load balancing, and rolling updates**. These capabilities allow the system to automatically adjust to varying workloads and update the application without interrupting user access. Through this automated pipeline, the system ensures faster software delivery, reduced deployment errors, and improved reliability. The integration of **Docker, CI/CD pipelines, and Kubernetes orchestration** creates a scalable and efficient platform for deploying modern web applications.

1.3 System Design

The **System Design** of the Dockerized Entertainment React Service explains the overall architecture and structure used to develop, deploy, and manage the application efficiently. The system is designed using modern **DevOps practices** that combine version control, containerization, automated CI/CD pipelines, and container orchestration. This integrated design helps automate the entire application lifecycle, from development to deployment, while ensuring reliability and scalability.

In this system, the application source code is stored in a **GitHub repository**, which acts as the central platform for version control and collaboration. Developers can push updates or new features to the repository, and every change automatically triggers a **CI/CD pipeline** configured using tools such as **Jenkins or GitHub Actions**. The pipeline is responsible for building the React application, running automated tests to verify functionality, and creating a **Docker image** that packages the application along with all necessary dependencies.

After the Docker image is built successfully, it is stored in a **container registry** such as Docker Hub or GitHub Container Registry.



The **Kubernetes cluster** then retrieves this image and deploys it across its nodes. Kubernetes manages the containers by providing features such as **automatic scaling, load balancing, health monitoring, and rolling updates**, which help maintain system stability and performance even during high user traffic or system failures.

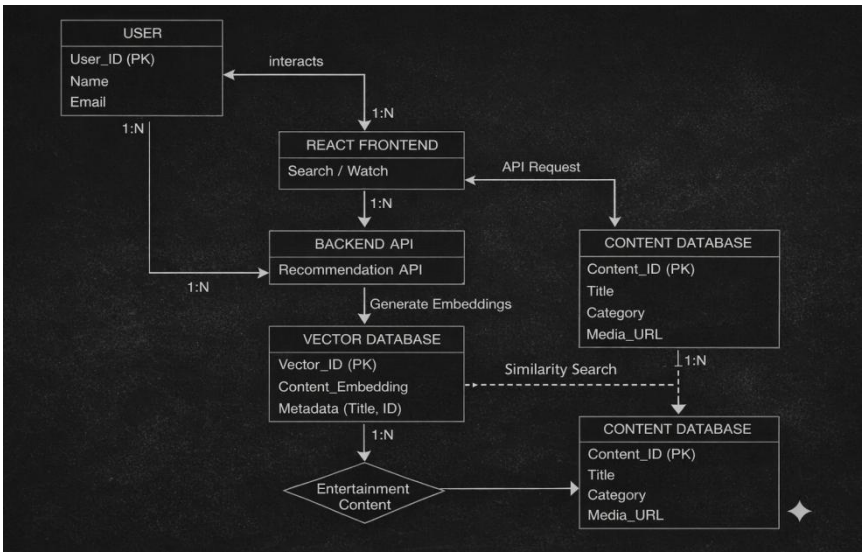
Overall, the system design creates a **fully automated, scalable, and efficient deployment environment**. By integrating Docker containerization, CI/CD automation, and Kubernetes orchestration, the system ensures consistent application performance, faster deployment cycles, and improved reliability for modern web applications.

1. Data Design

The **Data Design** of the Dockerized Entertainment React Service focuses on how application data, deployment information, and service configurations are stored and managed within the system. The system uses a **Vector Database** to efficiently store and retrieve data related to the entertainment platform. Vector databases are designed to handle high-dimensional data and enable faster search and recommendation capabilities, which are useful for applications that involve media content, search, or recommendation features.

In this system, the vector database stores information in the form of **vector embeddings**, allowing the platform to perform similarity searches and intelligent content retrieval. This helps in organizing entertainment content such as movies, shows, or media information in a structured way while enabling faster data access and improved user experience. The application interacts with the vector database through backend services to retrieve relevant data based on user queries or application requirements.

Additionally, deployment-related data such as **Docker images, CI/CD build records, and Kubernetes container information** are managed within the deployment infrastructure. Kubernetes monitors the running services and ensures that the application components are functioning correctly. By integrating a **vector database with containerized deployment**, the system provides efficient data handling, scalability, and high performance for modern entertainment applications.



3. Interfaces

The **Dockerized Entertainment React Service** provides multiple interfaces that allow developers and administrators to interact with the system, manage deployments, and monitor application performance. These interfaces help simplify the development workflow and ensure smooth communication between different components of the DevOps pipeline.

3.1 User Interface:

The main user interface is the CI/CD dashboard provided by Jenkins or GitHub Actions, which allows users to monitor and manage the automated pipeline. Through this dashboard, developers can track the status of application builds, observe deployment progress, and review logs generated during each stage of the pipeline. The interface also helps identify errors during the build or deployment process, making it easier to troubleshoot and maintain the system.

Additionally, the dashboard provides a visual representation of the workflow, which improves transparency and control over the deployment process.

3.2 Command Line Interface (CLI):

Developers also interact with the system using command-line tools such as Git CLI, Docker CLI, and Kubernetes CLI . Git CLI is used to manage source code, including pushing updates, pulling changes, and maintaining version control in the repository. Docker CLI is used to build Docker images, run containers, and manage containerized versions of the React application. Kubernetes CLI allows developers to deploy, scale, and monitor containers within the Kubernetes cluster. These command-line interfaces provide flexibility and allow developers to efficiently manage different stages of the development and deployment lifecycle.

2. State and Session Management

In the **Dockerized Entertainment React Service**, state and session management are important for maintaining the stability and reliability of the application during deployment and execution. The system ensures that application states and deployment configurations are consistently maintained throughout the DevOps pipeline. The deployment state of the application is primarily managed by **Kubernetes**, which continuously monitors the running containers and ensures that the desired number of instances are active. If a container fails or stops responding, Kubernetes automatically restarts or replaces it to maintain the required system state. This self-healing capability helps maintain high availability and system reliability.

Session management for users interacting with the application is handled through the **React frontend and backend services**, where user sessions are maintained during interactions with the platform. The system ensures that user sessions remain stable during updates by using Kubernetes features such as **rolling updates**, which allow new versions of the application to be deployed without interrupting active users.

Overall, the integration of Kubernetes orchestration and application-level session handling ensures smooth operation, consistent system state, and uninterrupted user experience within the Dockerized Entertainment React Service.

4. Functional Requirements

The **functional requirements** define the main operations and services that the **Dockerized Entertainment React Service** must perform in order to support automated development, containerization, and deployment of the application. These requirements describe how the system should behave and how different components interact to ensure smooth application delivery using DevOps practices.

4.1 Application and Repository Management

- **Code Management:** The system must allow developers to upload, update, and maintain the React application source code using a **GitHub repository**. This repository acts as the central storage location for all project files including application code, Docker configuration, and deployment scripts.
- **Version Control:** The system must support version control so that developers can track changes made to the application. This helps maintain a history of updates and allows developers to revert to previous versions if errors occur.
- **Secure Access:** Only authorized developers should be allowed to push or modify code within the repository. This ensures the security and integrity of the application source code.

4.2 CI/CD Pipeline Management

- **Automatic Pipeline Trigger:** Whenever developers push new code or updates to the repository, the system must automatically trigger a **CI/CD pipeline** using tools such as Jenkins or GitHub Actions.
- **Automated Build Process:** The pipeline must automatically build the React application without requiring manual intervention. This includes compiling the application and preparing it for deployment.
- **Pipeline Monitoring:** Developers and administrators must be able to monitor the status of the pipeline through the CI/CD dashboard, which displays build progress, logs, and deployment results.

4.3 Docker Containerization

- **Docker Image Creation:** The system must automatically generate a **Docker image** during the build stage of the pipeline. This image contains the application code and all required dependencies.
- **Environment Consistency:** Docker must ensure that the application runs consistently across development, testing, and production environments.
- **Image Storage:** The Docker image created during the build process must be pushed to a **container registry** such as Docker Hub or GitHub Container Registry so that it can be accessed during deployment.

4.4 Kubernetes Deployment and Orchestration

- **Automated Deployment:** Kubernetes must automatically retrieve the latest Docker image from the container registry and deploy it within the cluster environment.
- **Container Management:** The Kubernetes cluster must manage running containers and ensure that the application remains available at all times.

- **Auto-Scaling:** The system must support scaling of containers based on user demand or system load, allowing the application to handle increased traffic efficiently.
- **Service Exposure:** Kubernetes must provide networking and service exposure so that users can access the entertainment application through a web interface.

4.5 CI/CD Strategy (Continuous Integration and Continuous Deployment)

The CI/CD strategy defines the automated workflow that connects development activities with deployment of the application.

4.5.1 Continuous Integration (CI)

- **Automated Testing:** Every code update must trigger automated checks to detect errors or issues in the React application.
- **Code Validation:** The pipeline must verify that the application builds successfully and meets the required configuration before proceeding to deployment.
- **Artifact Creation:** Once the tests pass successfully, the system must create a Docker image that acts as a **stable deployment artifact** for the application.

4.5.2 Continuous Deployment (CD)

- **Container Deployment:** The Docker image must be automatically deployed to the **Kubernetes cluster** after the CI stage is completed.
- **Rolling Updates:** The system must support rolling updates so that new versions of the application can be deployed without interrupting the existing service.
- **High Availability:** The deployment process must ensure that the application remains accessible to users even while updates or scaling operations are taking place.

Overall, these functional requirements ensure that the **Dockerized Entertainment React Service** supports automated development workflows, reliable container deployment, and efficient management of application services.

5. Non-Functional Requirements

The **non-functional requirements** describe the quality, performance, and reliability aspects that the **Dockerized Entertainment React Service** must maintain. The system must ensure **secure access** to the GitHub repository, CI/CD pipeline, and container registry so that only authorized developers can manage the application and deployment process. Sensitive information such as API keys, tokens, and credentials must be stored securely using environment variables or secret management tools.

Secure communication between Docker, Kubernetes, and CI/CD services must also be maintained to protect system data and prevent unauthorized access.

In addition to security, the system must provide **high performance, scalability, and reliability**. The CI/CD pipeline should support fast build and deployment processes by using optimized Docker images and caching mechanisms. Kubernetes must efficiently manage system resources such as CPU and memory to ensure smooth operation of the containers. The system should also support **horizontal scaling**, allowing additional container instances to be created automatically when user demand increases. Furthermore, Kubernetes should ensure **high availability** by automatically restarting failed containers and supporting **rolling updates**, allowing new versions of the application to be deployed without causing downtime. Proper system design and documentation also ensure that the platform remains easy to maintain and update in the future.

6. References

• Docker Documentation:

The official Docker documentation provides information about containerization, including how to create Docker images, write Dockerfiles, and manage containers. It explains how Docker helps package applications with their dependencies to ensure consistent execution across different environments.

• Kubernetes Documentation:

The Kubernetes documentation explains how to deploy, manage, and scale containerized applications. It covers important concepts such as pods, services, deployments, and auto-scaling, which help maintain application availability and reliability.

• GitHub Documentation:

GitHub documentation provides guidance on version control using Git. It explains repository management, code collaboration, branching, and tracking code changes, which are important for managing the project source code.

• Jenkins / GitHub Actions Documentation:

These documentations explain how to configure CI/CD pipelines for automating application builds, testing, and deployment whenever new code is pushed to the repository.

- **React.js Documentation:**

The React.js documentation provides information about building user interfaces using components, managing application state, and structuring frontend applications effectively.

7. **Git Hub:** https://github.com/Mrunali-Kamerikar/Dockerized-Entertainment-React-Service